

LEAFMATE: PLANT DISEASE DETECTION AND MANAGEMENT APPLICATION

BY

UNIQUE GAURAV GAMAL

T.U. REGISTRATION NO: 7-2-551-175-2022

SYMBOL NO: 14756/22

National College of Computer Studies

A Project Report Submitted To

Faculty of Management, Tribhuvan University

In partial fulfillment of the requirements for the degree of

Bachelor of Information Management (BIM)

Paknajol, Kathmandu

February 2026

STUDENT DECLARATION

This is to certify that I have completed the Project entitled “**LeafMate: Plant Disease Detection and Management Application**” under the guidance of “Mr. Teksan Gharti Magar” in partial fulfillment of the requirements for the degree of Bachelor of Information Management at Faculty of Management, Tribhuvan University. This is my original work, and I have not submitted it earlier elsewhere.

Signature

Name: Unique Gaurav Gamal

Date: 4th February 2025

T.U. Registration No.: 7-2-551-175-2022

CERTIFICATE FROM THE SUPERVISOR

This is to certify that the project entitled “**LeafMate: Plant Disease Detection and Management Application**” is an academic work done by “Unique Gaurav Gamal” submitted in the partial fulfillment of the requirements for the degree of Bachelor of Information Management at Faculty of Management, Tribhuvan University under my guidance and supervision. To the best of my knowledge, the information presented by him in the project report has not been submitted earlier.

Signature of the Supervisor

Name: Teksan Gharti Magar

Designation: Mentor/Supervisor

Date: 4th February 2025

APPROVAL SHEET

This is to certify that the project titled “**LeafMate: Plant Disease Detection and Management Application**” submitted by “Unique Gaurav Gamal” has been examined and approved. In our opinion, it meets the required scope and quality standards for a project submitted in partial fulfillment of the requirements for the degree of Bachelor of Information Management (BIM).

Approval Panel:

S. No.	Name	Designation	Signature
1	Teksan Gharti Magar	Project Supervisor	
2	Rajan Poudel	Program Coordinator	
3	Nabaraj Bahadur Negi	Internal Examiner	
4	Sudip Raj Khadka	External Examiner	

Date of Defense: 4th February 2025

Department: Faculty of Management

Faculty: Bachelor of Information Management (BIM)

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who supported me throughout the project. First and foremost, I extend my heartfelt thanks to my supervisor for his invaluable guidance and support.

I am grateful to the faculty members of the National College of Computer Studies for providing a nurturing academic environment. Special thanks to my family and friends whose encouragement and understanding were essential during this journey.

Lastly, I appreciate the developers and community contributors of open-source technologies like Tensorflow, Python, Colab & Flutter which played a crucial role in making this project possible.

Unique Gaurav Gamal

T.U. REGISTRATION No: 7-2-551-175-2022

SYMBOL No: 14756/22

February, 2026

ABSTRACT

This project presents an offline mobile application designed to help farmers and home gardeners detect and manage plant diseases using a lightweight MobileNetV2-based convolutional neural network, converted into a TensorFlow Lite model (float16 quantized) for fast, on-device AI predictions. The application allows users to capture leaf images using their phone camera or upload them from the gallery. These images are preprocessed by cropping, resizing to 224×224 pixels, and normalizing, so the AI model can analyze them accurately. Once analyzed, the system provides instant results, including the disease name, confidence score, possible causes, symptoms, preventive measures, and suggested treatments, helping users take timely action to protect their plants. In addition to disease detection, the application supports comprehensive plant management by storing scan history in a local SQLite database, enabling users to track the health of their crops over time and monitor recurring disease patterns. Users can set reminders and notifications for treatments, ensuring that they do not miss important interventions for plant care. The app also includes multi-language support in English and Nepali, making it accessible to a wider audience, and allows users to customize themes for better usability. Its intuitive interface, powered by GoRouter for navigation and Riverpod for state management, ensures a smooth and user-friendly experience. The development followed iterative methods, involving repeated building, testing, and refinement, which helped achieve reliable performance and accurate results even on low-end devices. Overall, this application provides a practical tool for farmers and gardeners in Nepal and similar regions, empowering them to detect, track, and manage plant diseases efficiently. By combining offline AI predictions with reminders, history tracking, and clear guidance on treatment and prevention, the app promotes sustainable agriculture, reduces crop losses, and supports healthier, more productive farms. Its lightweight design, ease of use, and offline functionality make it particularly suitable for rural and resource-constrained areas, where access to agricultural experts or labs is limited. The project demonstrates how AI-powered mobile applications can transform traditional farming practices by providing timely, actionable insights directly to the hands of farmers and gardeners.

Keywords: Plant Disease Detection, Offline AI, MobileNetV2, TensorFlow Lite, Flutter, Image Preprocessing, Multi-language Support, Local Notifications, Riverpod, SQLite,

TABLE OF CONTENTS

STUDENT DECLARATION.....	ii
CERTIFICATE FROM THE SUPERVISOR.....	iii
APPROVAL SHEET.....	iv
ACKNOWLEDGEMENTS.....	v
ABSTRACT.....	vi
TABLE OF CONTENTS.....	vii
LIST OF FIGURES.....	ix
LIST OF TABLES.....	x
ABBREVIATIONS.....	xi
CHAPTER I.....	1
INTRODUCTION.....	1
1.1 Background of the Project.....	1
1.2 Problem Statement	2
1.3 Objectives.....	2
1.4 Review of Related Work and Literature	2
1.5 Development Methodology.....	3
1.6 Scope and Limitations.....	4
1.6.1 Scope:	4
1.6.2 Limitations:.....	4
1.7 Report Organization	5
CHAPTER II.....	6
SYSTEM DEVELOPMENT PROCESS	6
2.1 Analysis.....	6
2.1.1 Requirement Analysis.....	6
2.1.2 Feasibility Study	8
2.1.3 System Modelling (Object-Oriented Approach)	9
2.2 System Design.....	12
2.2.1 Refinement of Class.....	12
2.2.2 Component Diagram.....	14
2.2.3 Deployment Overview.....	14
2.3 Implementation.....	15
2.3.1 Tools Used.....	15
2.3.2 Module Description	15
2.3.3 Testing	17

CHAPTER III	20
CONCLUSION AND RECOMMENDATION	20
3.1 Summary	20
3.2 Conclusion.....	20
3.3 Recommendation.....	20
REFERENCES.....	21
APPENDICES	a

LIST OF FIGURES

Figure 2. 1: Use-case diagram	6
Figure 2.2: Gant chart	9
Figure 2.3: Class Diagram	10
Figure 2.4: Sequence Diagram.....	11
Figure 2.5: Activity Diagram.....	12
Figure 2.6: Refined class diagram	13
Figure 2.7: Component diagram	14
Figure 2.8: Deployment diagram.....	14

LIST OF TABLES

Table 2.1 Tools Used	15
Table 2.2: Unit Testing	17
Table 2.3: System Testing.....	18

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DB	Database
E2E	End-to-End
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
ML	Machine Learning
SDK	Software Development Kit
TFLite	TensorFlow Lite
UI	User Interface

CHAPTER I

INTRODUCTION

1.1 Background of the Project

Agriculture is the backbone of many developing economies, including Nepal, where a large portion of the population depends on farming for livelihood. Plant diseases remain one of the major threats to agricultural productivity, often causing significant yield loss and economic hardship for farmers. Traditional disease identification methods rely heavily on expert consultation, which may not always be accessible or affordable for smallholder farmers in rural areas.

In this technologically advanced era, with the proliferation of artificial intelligence, many mobile apps are available for plant disease detection, diagnosis, and treatment, each with a variety of features. These apps need to be categorized and reviewed following a proper framework that ensures their quality. This study aims to present an approach to evaluating plant disease detection mobile apps, which includes providing ratings of distinct features of the apps and insights into the exploitation of artificial intelligence used in plant disease detection (Siddiqua, 2022).

Plants play an important role in agriculture, home gardening, and food production. However, plant diseases are a common problem that affects crop quality, plant health, and productivity. Many farmers and home gardeners find it difficult to identify plant diseases early because it usually requires expert knowledge or access to agricultural support services.

With the growth of smartphones and artificial intelligence, it is now possible to build simple tools that help users detect plant diseases using images of plant leaves. This project, LeafMate, is designed to provide an easy way for users to detect plant diseases and manage plant health using a mobile application.

The application allows users to capture or upload a photo of a plant leaf, after which a trained AI model analyzes the image and predicts possible diseases. In addition to detection, the system also supports basic plant management features, such as viewing disease information, receiving care suggestions, and maintaining a simple scan history for tracking plant health over time.

This system can be useful for farmers who need quick disease identification in crops to prevent damage and improve productivity. It can also support home gardeners who want to take better care of their plants by helping them recognize possible diseases and understand basic plant care. In addition, the application can be helpful for students or

beginners who are learning about plant health and disease identification in a simple and practical way.

By combining AI-based disease detection with simple plant management support, the system aims to make plant care more accessible, faster, and easier without requiring specialized knowledge.

1.2 Problem Statement

Plant diseases can cause serious crop loss and affect food production, especially in areas where farmers do not have easy access to agricultural experts or laboratories. Traditional methods of identifying plant diseases can be slow, costly, and difficult to use in everyday farming. Recent developments in artificial intelligence and machine learning, especially image-based models, make it possible to detect plant diseases from leaf images. However, many existing systems depend on large models or internet access, which makes them less practical for farmers and home gardeners in rural or offline environments.

1.3 Objectives

The project aims to achieve the following objectives:

- To develop a cross-platform mobile application that allows users to capture or upload plant leaf images and detect diseases using an AI-based model that can run offline.
- To help farmers and home gardeners manage plant health by providing disease information, basic treatment suggestions, and reminder notifications within the application.

1.4 Review of Related Work and Literature

In recent years, there has been a growing application of ML and DL in agriculture, particularly for plant disease detection. Traditional methods, such as expert consultation, field surveys, or laboratory analysis, are often time-consuming, costly, and inaccessible for farmers in remote regions. With the availability of large-scale agricultural datasets and advances in computer vision, researchers have developed AI-driven solutions that enable faster and more accessible disease diagnosis.

A study published in *Agronomy* (Siddiqua, 2022) demonstrated that CNNs can achieve strong performance in identifying plant diseases from leaf images, sometimes exceeding human-level accuracy under controlled conditions. The study highlighted the importance of transfer learning architectures like MobileNet, EfficientNet, and ResNet, which provide lightweight yet effective models suitable for mobile devices and resource-constrained environments. However, challenges such as class imbalance in datasets and domain shift from controlled to real-world images remain.

Similarly, research in the LBEF Journal of Science and Technology (Singh S., 2020) explored practical ML approaches for disease detection in common crops. The study discussed traditional ML classifiers such as Support Vector Machines (SVMs) and Random Forests, alongside CNN-based models, illustrating the evolution toward end-to-end deep learning pipelines. The work also emphasized the need for localized datasets that reflect region-specific crops and disease variations, a critical factor for real-world applicability in Nepalese agriculture and other developing countries.

Overall, the literature indicates that while CNN-based models are highly effective in controlled experiments, practical deployment for farmers requires addressing dataset imbalance, limited device capacity, offline inference capability, and region-specific disease variations. Building on these insights, this project develops an offline mobile AI application using MobileNetV2 and other lightweight architectures, aiming to help farmers and home gardeners identify diseases in crops such as tomato, potato, maize, apple, and grapevine, with features like image preprocessing and offline predictions for accessibility and usability.

1.5 Development Methodology

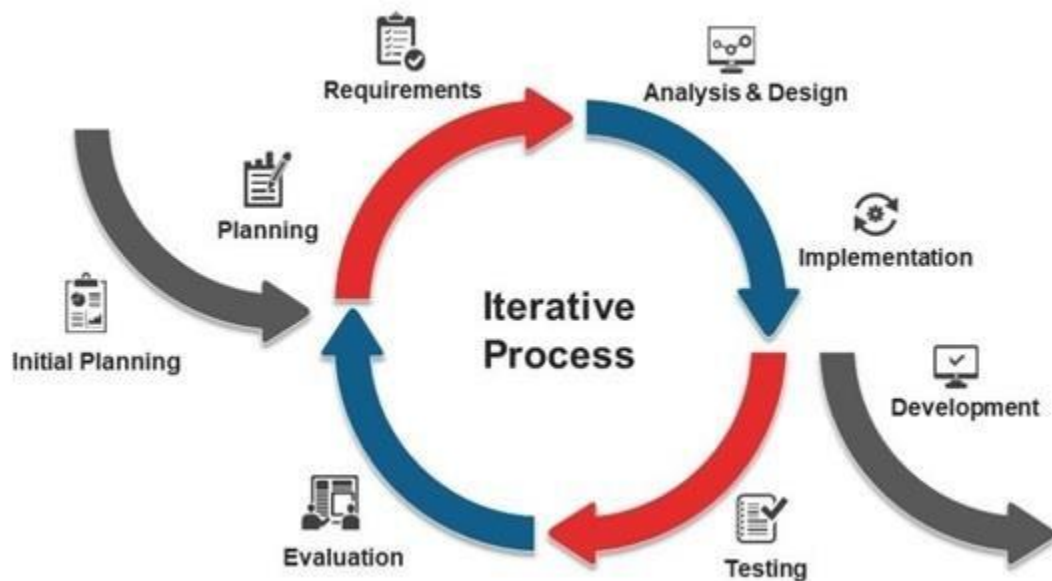


Figure 1. 1: Iterative development model (Miraz, 2020)

This project follows an iterative development approach, which means the system is built gradually through repeated cycles of creating, testing, and improving different parts. Each cycle focuses on making the app better and adding new features step by step. Key areas of improvement include capturing and preprocessing leaf images, detecting plant diseases using AI, storing scan history, setting reminders and notifications, supporting multiple languages, and refining app settings for a smooth user experience.

This approach is useful because it allows developers to make changes based on feedback, fix issues early, and continuously improve the system. After each cycle, manual testing is done through the app itself to make sure everything works correctly. Users can capture or upload leaf images, run AI predictions, check their scan history, set reminders, and change app settings without any problems.

Manual testing also ensures that AI predictions are accurate, notifications appear at the right time, language settings switch properly, and scan data is saved correctly in the database. By using this step-by-step method, the app becomes reliable, user-friendly, and practical for farmers, home gardeners, and plant enthusiasts.

1.6 Scope and Limitations

1.6.1 Scope:

The project focuses on developing an offline mobile AI application that enables users to capture or upload leaf images and identify diseases in common crops. The system is designed with a user-friendly interface for farmers and home gardeners, providing quick and actionable insights on plant health. Core functionalities include image capture and selection, basic preprocessing such as cropping and resizing, and disease prediction using lightweight CNN models (e.g., MobileNetV2) optimized for offline mobile inference. The application delivers detailed information on disease causes and suggested treatments, helping users take timely action. It also supports multi-language output (English and Nepali), maintains a scan history, and allows users to set reminders and notifications for treatment schedules. By integrating AI-driven predictions with practical guidance, the system empowers users to monitor and manage plant health efficiently, improving crop care and productivity at the household or small-farm level.

1.6.2 Limitations:

The system has certain limitations that define its current scope and functionality. It primarily focuses on plant disease detection for selected crops and may not correctly identify diseases outside the trained classes. The AI model is quantized for mobile deployment, which enables offline inference but slightly reduces prediction precision, occasionally resulting in misclassification. The application relies on clear and focused leaf images, so blurry or poorly captured images can affect accuracy. Performance may vary across devices with limited computational capacity. Notifications and reminders depend on device permissions and settings, which may affect timely delivery. Additionally, the dataset used may not fully represent region-specific disease variations, which can constrain the app's applicability in certain local contexts.

1.7 Report Organization

This report is structured to document the development, implementation, and completion of the offline plant disease detection mobile application. The chapters are organized as follows:

- **Chapter 1: Introduction**

This chapter provides the background of the project, the problem statement, objectives, development methodology, scope, and limitations. It outlines the motivation for creating an AI-powered, offline solution for identifying plant diseases and sets the context for the application. The chapter highlights the need for accessible plant health tools for farmers and home gardeners, and explains the incremental approach used to build the system.

- **Chapter 2: System Development Process**

This chapter details the structured development process, including requirement analysis, feasibility study, and dataset collection. It describes the design and implementation of the AI model and mobile application, covering image preprocessing, model selection (MobileNetV2), quantization, and Flutter integration. The chapter also documents the implementation of features such as scan history, reminders, multi-language support, notifications, and UI screens, along with testing approaches conducted to ensure usability and correctness.

- **Chapter 3: Conclusion and Recommendations**

This chapter summarizes the successful development of the fully functional LeafMatelication, discussing key achievements, challenges, and observed limitations. It highlights how the system provides offline disease predictions, practical treatment suggestions, and a user-friendly interface for crop monitoring. The chapter concludes with recommendations for enhancing model accuracy, expanding crop and disease coverage, and improving app usability for future iterations.

CHAPTER II

SYSTEM DEVELOPMENT PROCESS

2.1 Analysis

2.1.1 Requirement Analysis

I. Functional Requirement:

- Use Case Diagram

The use case diagram shows how users capture or upload leaf images to detect plant diseases using the app. It also includes viewing prediction results, managing scan history, setting reminders, and changing app settings.

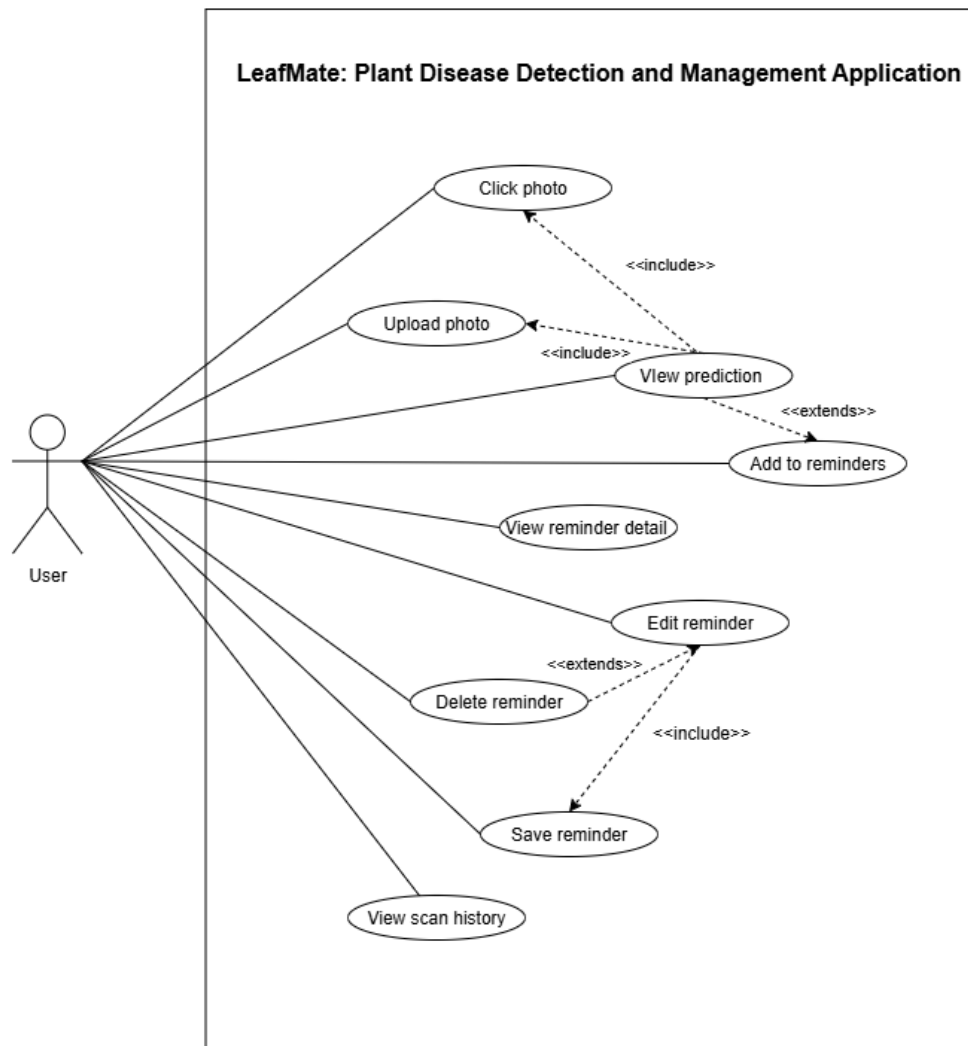


Figure 2. 1: Use-case diagram

II. Non-functional Requirement:

The non-functional requirements define the quality attributes, constraints, and operational characteristics of the offline plant disease detection mobile application. These requirements ensure that the system is reliable, efficient, usable, and suitable for deployment in real-world agricultural environments.

- **Performance**

The application is optimized for offline operation on mobile devices with limited computational resources. AI inference using the TensorFlow Lite–based MobileNetV2 model produces prediction results within an acceptable response time after image capture or selection. Image preprocessing and prediction are designed to run efficiently without causing noticeable lag or application crashes on mid-range Android devices.

- **Usability**

The system provides a simple and intuitive user interface suitable for farmers, home gardeners, and non-technical users. Core functions such as image capture, disease scanning, viewing results, accessing scan history, and setting reminders are easily accessible with minimal user interaction. Icons, labels, and layouts are designed to reduce learning effort and support smooth navigation.

- **Reliability**

The application operates fully offline once installed, ensuring consistent functionality even in areas with poor or no internet connectivity. Stored scan history, reminders, and settings persist across application restarts through local database storage. The system handles invalid or unclear images gracefully by providing appropriate feedback to the user.

- **Accuracy and Consistency**

The AI model provides consistent prediction outputs for similar input images under controlled conditions. While accuracy is constrained by dataset limitations and quantization effects, the system reliably produces a disease classification with confidence values and corresponding informational content (causes and treatments).

- **Portability**

The application is developed using Flutter, allowing compatibility across multiple Android devices with varying screen sizes and hardware

configurations. The use of TensorFlow Lite ensures that the AI model is portable and suitable for mobile deployment.

- **Security and Privacy**

All data processing, including image analysis and disease prediction, is performed locally on the user's device. No images or personal data are transmitted to external servers, ensuring user privacy and data security. Application permissions are limited to camera and storage access only when required.

- **Maintainability**

The system follows a modular architecture separating models, services, providers, and user interface components. This structure simplifies debugging, updates, and future enhancements. Clear separation of concerns allows individual components such as the AI model, database service, or UI screens to be modified without affecting the entire system.

2.1.2 Feasibility Study

- **Technical**

The project leverages TensorFlow, TensorFlow Lite, Flutter, and Google Colab, which are compatible with mobile development and offline inference. Google Colab was used for training and experimenting with CNN models (MobileNetV2, MobileNetV3, EfficientNet Lite) using GPU resources available in the cloud. The availability of public datasets such as PlantVillage and Kaggle provides sufficient data for initial model training. This ensures that the AI model can be implemented and deployed on mobile devices without requiring high-end local hardware.

- **Operational**

The application targets farmers and home gardeners, providing a simple and accessible tool for plant disease detection. Offline functionality ensures usability in remote areas without stable internet access. The system is designed to be user-friendly, with basic image selection, cropping, and disease prediction features.

- **Economic**

Development costs are minimal as the project is a solo academic prototype. It relies on open-source frameworks, Google Colab for cloud-based training, and free datasets, making it feasible within the limited time and resource constraints of the project.

- **Schedule**

The project timeline is represented using a Gantt chart, showing the planned iterations, tasks, and durations over the course of 1.5 months. The schedule follows an iterative development methodology, allowing incremental progress and continuous refinement of both the AI model and the mobile application.

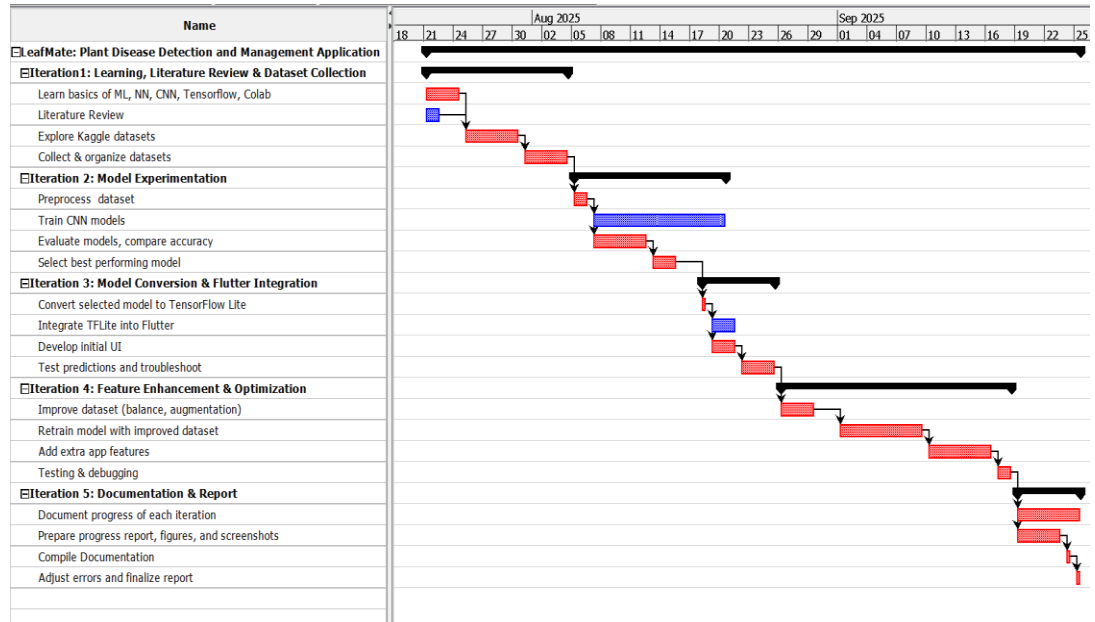


Figure 2.2: Gantt chart

2.1.3 System Modelling (Object-Oriented Approach)

- **Object Modelling (Class Diagram)**

The class diagram shows the main building blocks of the app and how they are connected.

It includes:

- CameraService takes photos from camera or gallery
- AiModel loads the AI model and predicts the plant disease
- PredictionResult holds the disease name and confidence score
- ScanHistoryModel saves details of past scans (photo path, disease, time, notes)
- ReminderModel stores reminder information (title, time, repeat option)
- ReminderProvider keeps track of all reminders
- NotificationHandler plans and cancels notifications
- DBService saves and loads scans and reminders to the phone's database

- HomeProvider remembers the latest prediction and list of recent scans

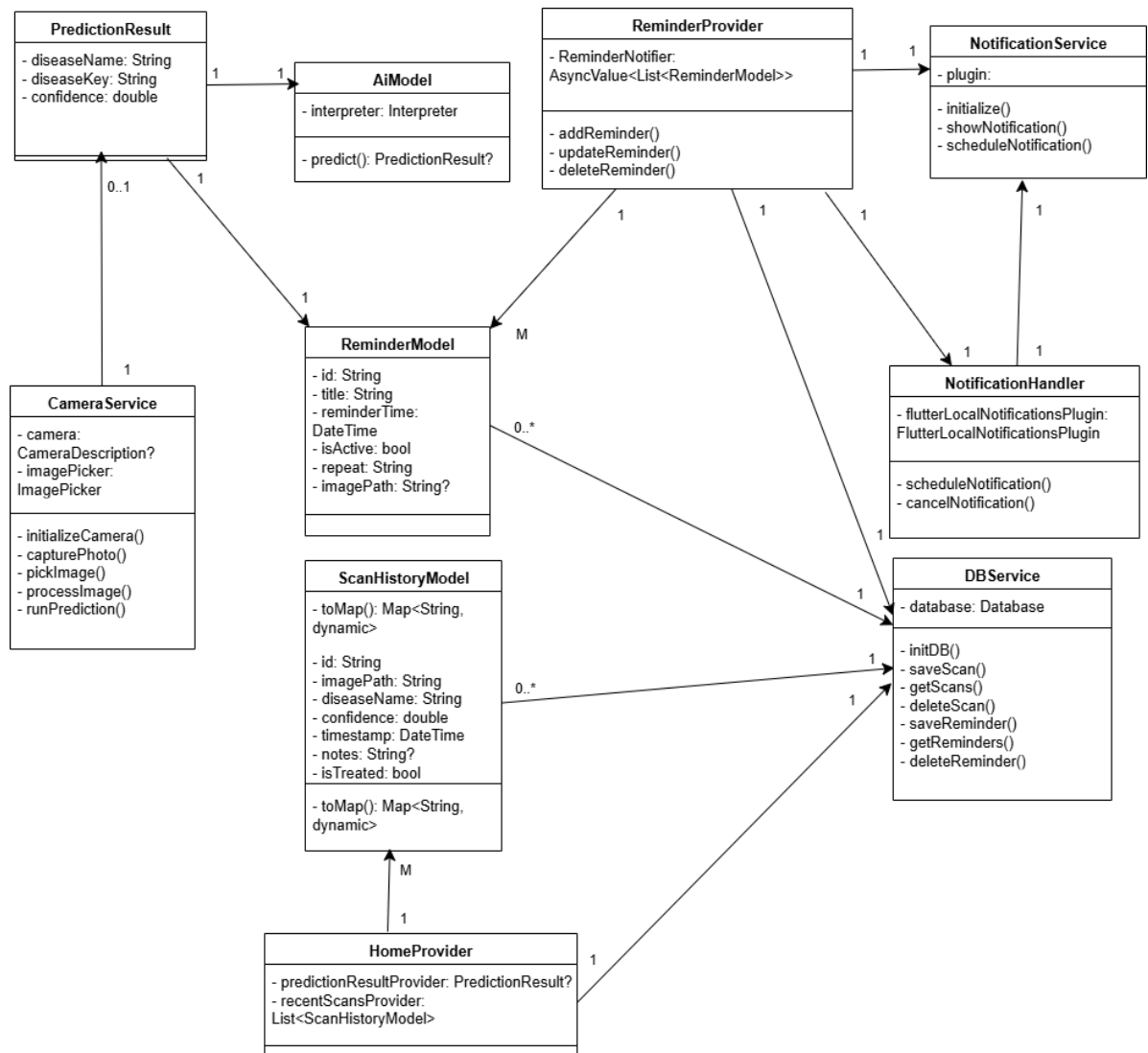


Figure 2.3: Class Diagram

- **Dynamic Modelling (State and Sequence Diagrams):**

- **Sequence Diagram:**

- User taps on scan button.
- User selects or captures an image.
- CameraScreen preprocesses the image (crop, resize, normalize).
- AiModel receives the preprocessed image and performs offline prediction.
- PredictionDisplay presents the disease name, confidence, and associated cure information to the user.
- DBService stores the result to the database.

- User reviews the results and may select another image, repeating the cycle.

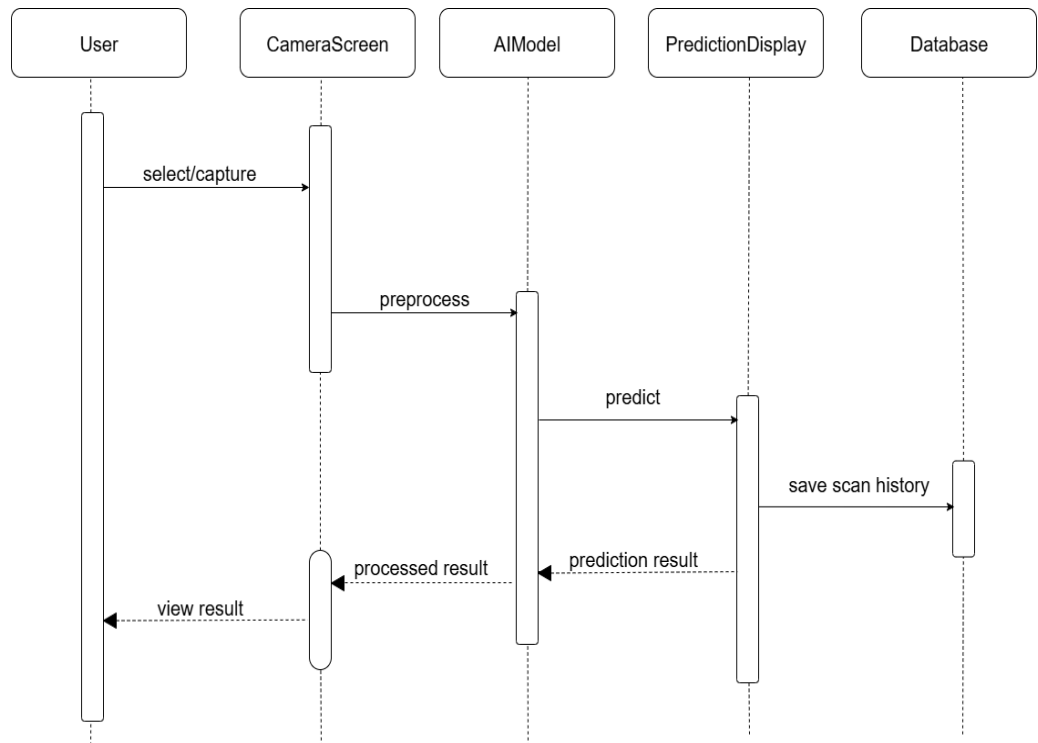


Figure 2.4: Sequence Diagram

- **Process modelling using Activity Diagrams**
 - **Activity Diagram:**

This activity diagram illustrates the main user flow of the LeafMate app, covering plant scanning, history review, and reminder management. On launch, users can start a new scan, view past scans, or access reminders. Scanning prompts camera or gallery selection, and the AI model identifies the plant and displays care instructions. Users can then perform another scan or create a care reminder. The history section shows past identifications with detailed care information and options to add or edit reminders. The reminders section lets users view, create, or modify care tasks, with notifications scheduled to ensure timely alerts. This flow enables seamless plant identification, care tracking, and routine management.

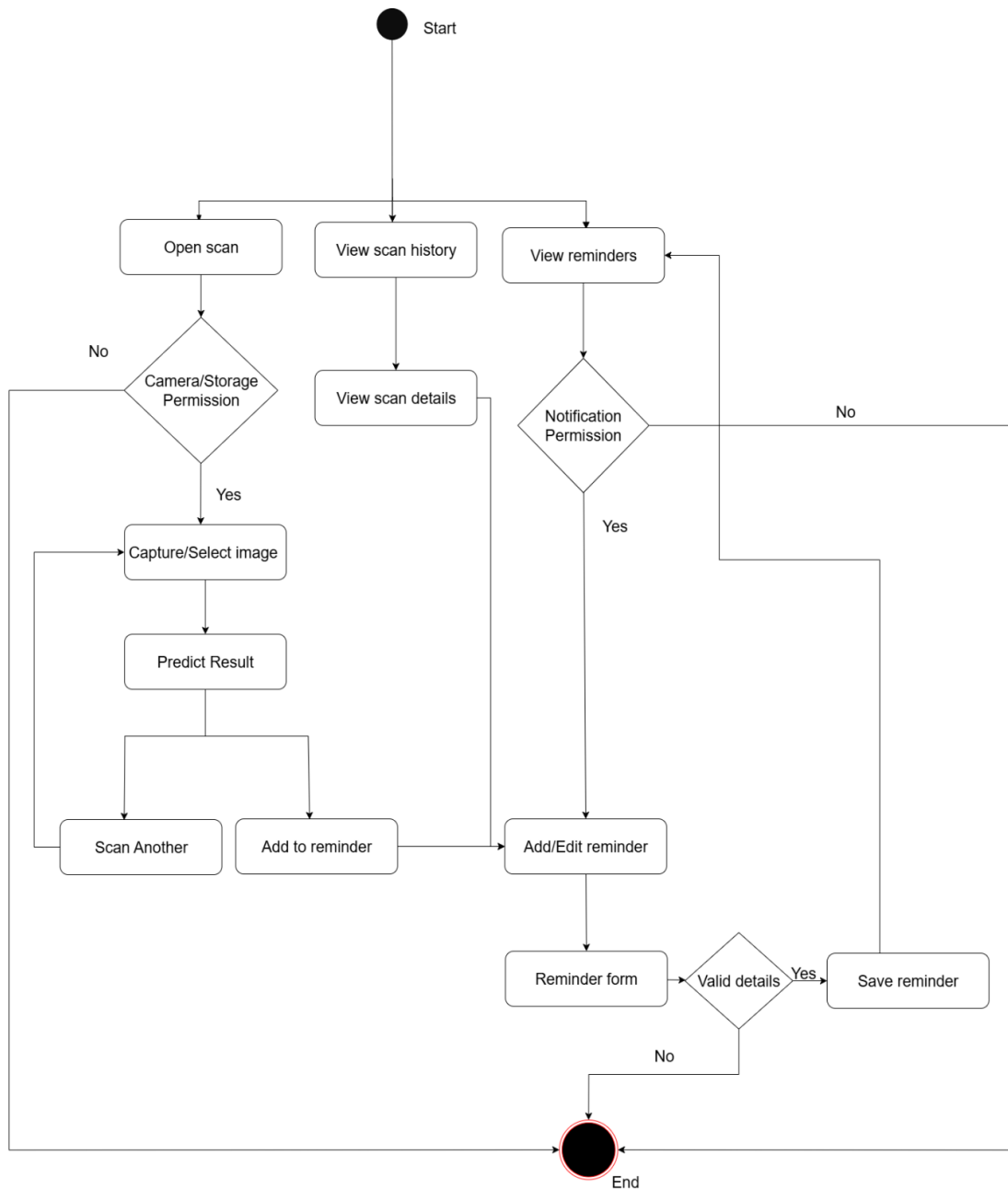


Figure 2.5: Activity Diagram

2.2 System Design

2.2.1 Refinement of Class

The refined class diagram shows the main parts of the app and how they connect clearly. It includes CameraService for taking photos, AiModel for predicting plant diseases, PredictionResult for showing the disease name and confidence, ScanHistoryModel for saving past scans, ReminderModel for reminder details, ReminderProvider for managing reminders, NotificationHandler for scheduling alerts, and DBService for storing everything in the database. These parts work together to handle photo capture, disease detection, saving scans, creating

reminders, and sending notifications. The diagram also highlights the interactions between these components, showing how data flows from user actions to the database and back. This clear structure ensures maintainability and makes it easier to extend the app with new features in the future.

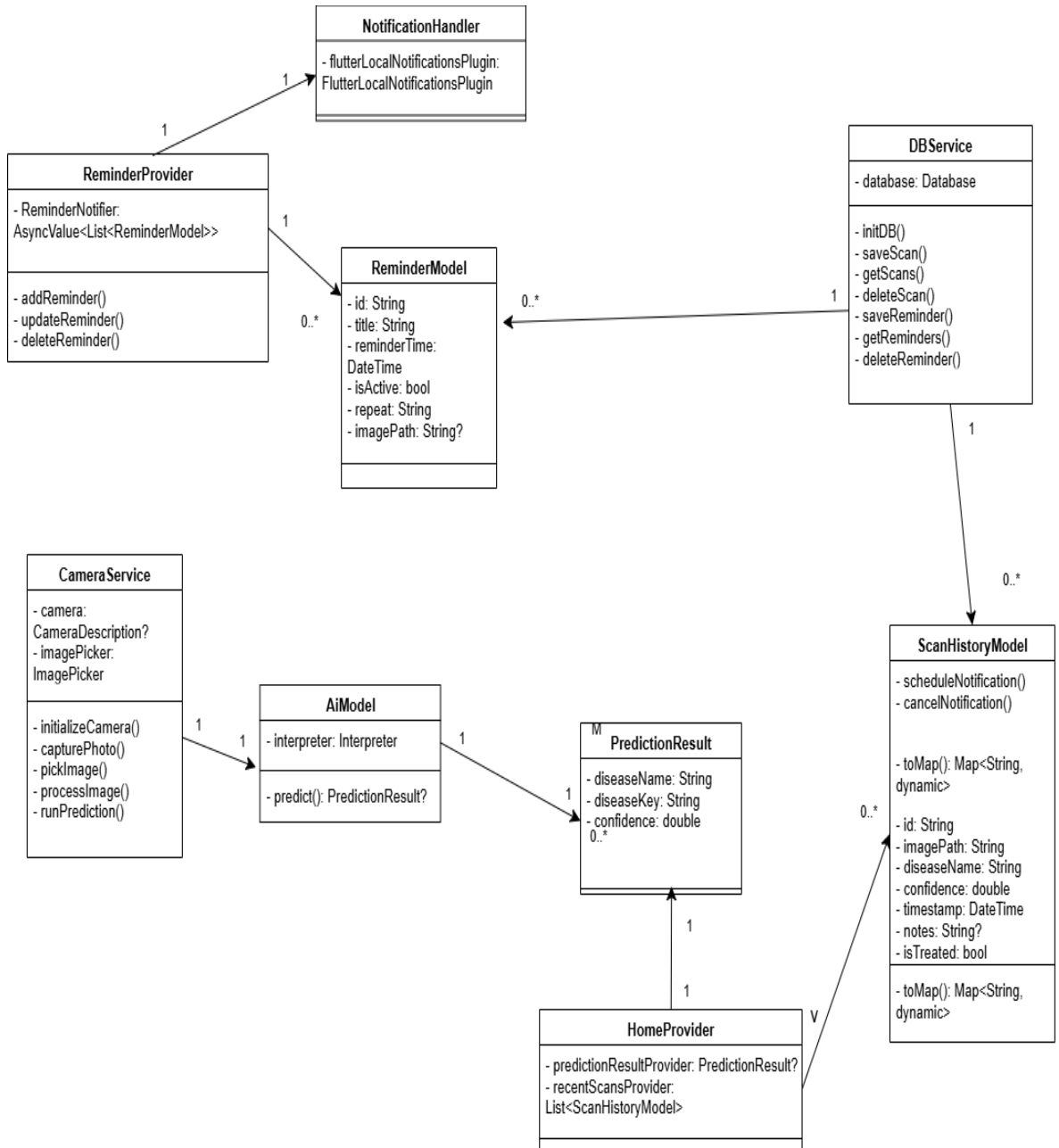


Figure 2.6: Refined class diagram

2.2.2 Component Diagram

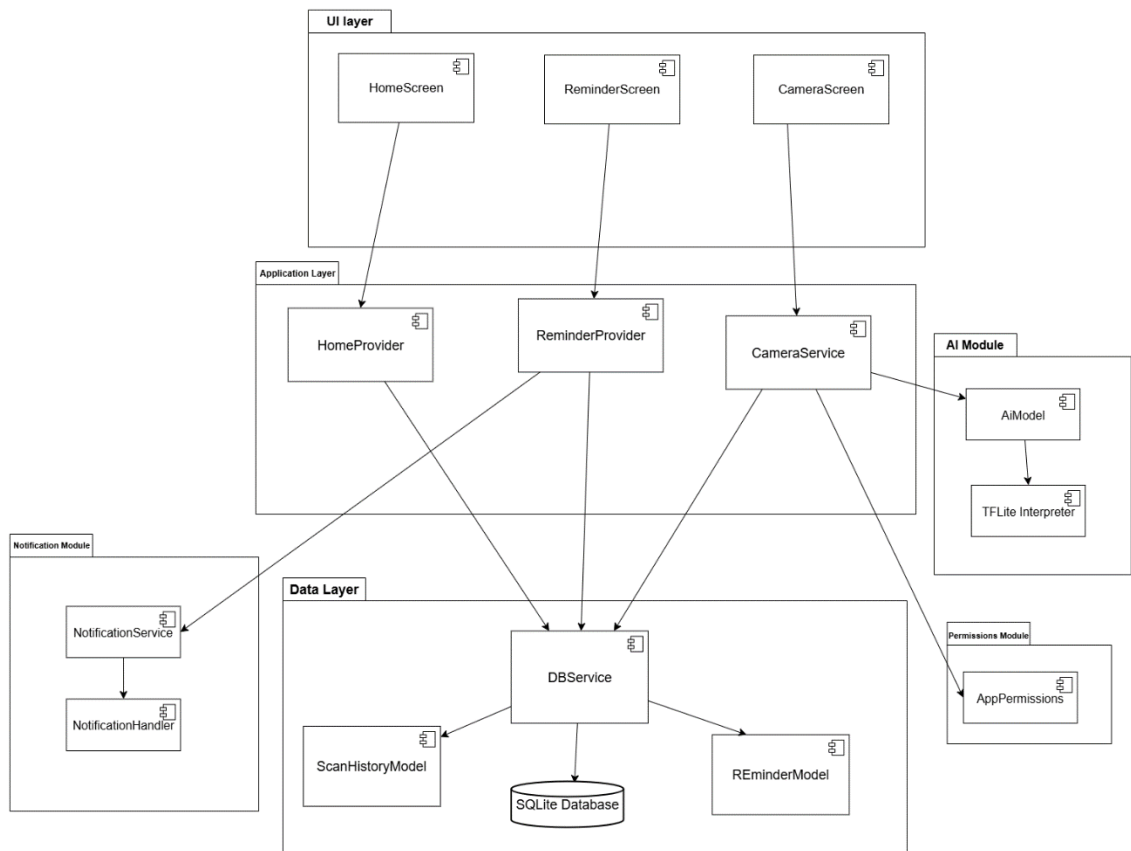


Figure 2.7: Component diagram

2.2.3 Deployment Overview

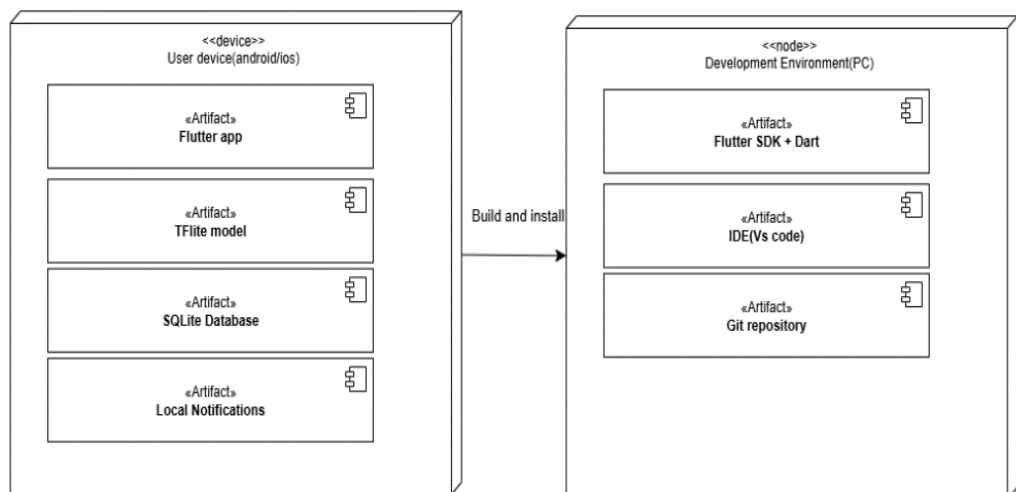


Figure 2.8: Deployment diagram

2.3 Implementation

2.3.1 Tools Used

Table 2.1: Tools Used

Component	Tools/Platforms	Use-case
Frontend	Flutter	Designing and implementing mobile UI
Model Training	TensorFlow, Python, Colab	Training CNN models (MobileNetV2)
Model Deployment	TensorFlow Lite	Offline inference on mobile devices
Database	SQLite	Saving reminder and scan history

2.3.2 Module Description

LeafMate is built with several core modules from the Flutter application structure, each responsible for specific functionality to ensure a smooth and efficient user experience:

- **Camera Service Module (`camera_service.dart`):**

This module manages all camera-related operations in the application. It allows users to capture plant leaf images using the device camera or select images from the gallery. After an image is selected, the module performs basic preprocessing steps such as cropping and preparing the image for AI prediction. It also connects the image capture process with the home screen state and AI prediction system.

- **AI Model Module (`ai_model.dart`):**

This module is responsible for loading the TensorFlow Lite MobileNetV2 model and running plant disease predictions. It receives processed image data and performs inference using the TFLite interpreter. The module returns prediction results, including the detected disease name and confidence level. Since the model runs locally on the device, disease detection works even without internet access.

- **Database Service Module (`db_service.dart`):**

This module handles local database operations using SQLite. It stores scan history records and reminder data so that users can access them later. The module provides functions for creating, reading, updating, and

deleting scan and reminder entries, allowing the app to maintain plant health records over time.

- **Reminder Module (reminder_provider.dart and notification_handler.dart):**

This module handles creating, editing, and deleting reminders for plant care activities, such as watering or treatment schedules. It uses local notifications to alert users at the specified time, even when the app is running in the background or closed. Reminders are stored in a local database, ensuring they persist across sessions, and users can easily modify or remove them as needed. This ensures that important plant care tasks are not missed and provides a smooth and reliable user experience.

- **Localization Module (localization_provider.dart and text_provider.dart):**

This module provides multi-language support for the application. It loads text content from JSON files stored in assets and displays the correct language based on user selection. This allows the application to support both English and Nepali users.

- **Settings Module (settings_provider.dart):**

This module manages application settings such as theme mode, language preference, notification permissions, and routing state. User preferences are stored locally using SharedPreferences so that settings remain saved when the app is reopened.

- **Home State Module (home_provider.dart):**

This module manages the main application state related to disease detection. It stores prediction results, image paths, status messages, and recent scan data. It also handles loading the AI interpreter and updating the UI when prediction results change.

- **Navigation History Module (navigation_history_provider.dart):**

This module keeps track of screen navigation during an app session. It stores the routes or pages visited by the user and helps maintain smooth movement between screens using the provider-based state management system. By remembering the navigation flow, it allows the back button and screen transitions to work correctly while the app is running. The stored navigation history exists only for the current session and resets when the application is closed, keeping the navigation behavior simple and efficient.

2.3.3 Testing

- **Unit Testing:**

Table 2. 2: Unit Testing

T ID	Test Case Description	Steps	Expected Output	Actual Output	Status
UT-01	Predict disease on image	1.Click/uoload image	Show prediction result	Prediction Result	Pass
UT-02	Crop image to focus on leaf area	1. Click/upload image 2. Crop to desired size	Cropped image (centered & resized ROI)	Cropped image	Pass
UT-03	Insert predicted result reminder	1. Click/upload image 2. Tap on add to reminder button 3. Save	Saved to database and loaded to reminder screen	Saved and loaded in reminder screen	Pass
UT-04	Add new reminder and schedule notification	1. Go to reminder screen 2. Tap on + button 3. Fill all fields 4. Save	Saved to database and loaded to reminder screen	Saved and loaded in reminder screen	Pass

Test Interpretation:

The unit testing results indicate that all core functionalities of the plant disease detection application are working as expected. The image prediction feature accurately processes uploaded images and provides correct disease predictions. Image cropping functions properly, allowing users to focus on the leaf area for more accurate analysis. The reminder system is fully operational, enabling users to save predicted results, create new reminders, and schedule notifications

without any errors. Overall, all tested units passed successfully, demonstrating that the individual components of the application are stable, reliable, and ready for integration into the complete system.

- **System Testing:**

Table 2.3: System Testing

T ID	Test Case Description	Steps / Input	Expected Output	Actual Output	Status
ST-01	Select image from gallery , preprocess, predict and display result	1. Open app and go to Home Screen 2. Tap "Gallery" button 3. Select valid leaf image	Prediction displayed (disease name, confidence, cure info), saved to history	Completed successfully	Pass
ST-02	Capture image, preprocess, predict and display result	1. Home Screen → Tap "Camera" button 2. Grant camera permission 3. Capture photo	Prediction shown, image processed (cropped/resized), result + cure info displayed, saved to DB	Completed successfully	Pass
ST-03	Scan, Save, View in History and View scan details	1. Perform successful scan (gallery/camera) 2. Navigate to History 3. Tap entry	History shows entry (thumbnail, disease, timestamp), detail screen shows full info	History & details displayed correctly	Pass
ST-04	Add reminder from scan detail and Verify notification	1. After scan, go to detail screen 2. Tap "Add Reminder" 3. Set title, time and Save	Reminder added, notification scheduled (appears at set time via local notifications)	Notification fires as expected	Pass
ST-05	First-time camera permission request & denial handling	1. Fresh install, go to Home and Tap Camera 2. Deny permission	App shows error message ("Camera access required") with retry option	Error message shown, retry works	Pass

ST-06	Scan workflow offline (airplane mode)	1. Enable airplane mode 2. Perform gallery/camera scan then predict and save reminder	AI inference works offline, prediction & save successful, local notification schedules	All features work offline	Pass
ST-07	Home to Drawer and Change language	1. Go to Home 2. Open Drawer and Change language	UI text updates to selected language, persists after restart	Language switches correctly	Pass
ST-08	Add recurring reminder , Close app, Trigger notification	1. Add reminder (daily repeat) 2. Close/kill app 3. Wait for trigger	Local notification appears, tapping opens app to Reminder View	Notification triggers & opens app	Pass
ST-09	Change theme (light/dark) and Verify UI updates	1. Open drawer and Toggle dark mode 2. Navigate to Home, History, Scan Detail	Entire app (drawer, cards, text) switches instantly to new theme	Theme applies globally	Pass

Test Interpretation:

The system testing results show that the LeafMate application functions correctly across all major workflows. Both gallery and camera scans produce accurate predictions and store results successfully. Users can view scan history, add and manage reminders, and receive notifications reliably. Permissions, offline functionality, navigation, language settings, and theme changes work as intended. Overall, all system test cases passed, confirming that the application is stable, reliable, and ready for real-world use.

CHAPTER III

CONCLUSION AND RECOMMENDATION

3.1 Summary

This project developed an offline AI-powered mobile application for plant disease detection using a lightweight MobileNetV2-based CNN model converted to TensorFlow Lite (float16 quantized) for efficient on-device inference. It has been built with Flutter which enables users to capture or select leaf images, preprocess them (crop, resize to 224×224, normalize), and instantly receive predictions including disease name, confidence score, causes, symptoms, prevention, and treatment recommendations for common crops. It operates fully offline, stores scan history in a local SQLite database, supports reminders with flutter_local_notifications, and offers multi-language interface (English/Nepali), theme switching, and navigation via GoRouter and Riverpod state management. Iterative development with thorough unit and system testing ensured reliable image processing, accurate inference, seamless offline functionality, and smooth user experience, making the app a practical, accessible tool for farmers and gardeners in remote areas with limited connectivity.

3.2 Conclusion

This project shows that it is possible to build a fully offline mobile app for plant disease detection using a lightweight AI model (MobileNetV2 converted to TensorFlow Lite). The app can quickly analyze leaf images on the phone, even without internet, and gives reliable predictions. It includes features like image preprocessing, scan history storage, reminders for treatment, multi-language support (English and Nepali), theme options, and easy navigation.

By using a lightweight model, the app balances speed, accuracy, and efficiency, making it suitable for farmers, home gardeners, and small-scale growers in rural areas. Users can identify plant diseases quickly and get helpful advice on symptoms, causes, and treatment. Overall, this project highlights how offline AI apps can support timely disease management, reduce crop losses, and promote better farming practices in places like Nepal.

3.3 Recommendation

Future improvements could include expanding the dataset with Nepalese crops and local disease types to improve accuracy, adding more languages and locally relevant treatment tips, and enhancing notifications for urgent cases. Optimizing the AI model for low-end devices and collecting user feedback would keep the app efficient and reliable. These updates would make the app more inclusive and useful for sustainable farming in Nepal and similar regions.

REFERENCES

- G., T. (2024). *Why Iterative Problem-Solving is important to improve ML models*. Medium. <https://medium.com/@gunjantitiya/why-iterative-problem-solving-is-important-to-improve-ml-models-1c837be05ce9>
- Miraz, D. M. (2020, January). *ResearchGate*. https://www.researchgate.net/figure/SDLC-Iterative-Model-2_fig4_338710620
- Siddiqua, A. K. (2022). *Evaluating Plant Disease Detection Mobile Applications: Quality and Limitations*. MDPI. <https://www.mdpi.com/2073-4395/12/8/1869>
- Singh S., & S. (2020). *2-1-119-137.pdf*. LBEF. <https://www.lbef.org/journal/2-1/download/2-1-119-137.pdf>

APPENDICES

